

JAVASCRIPT

Conceptos básicos explicados y resumidos

Alcance · funciones anónimas · callbacks · arrow functions · closures · spread · arreglos

Objetivo Condensar las 43 diapositivas en una guía de estudio corta, con la idea central y el ejemplo mínimo de cada concepto.

Basado exclusivamente en el material de la cátedra "Programación Orientada a Objetos 2".

Ing. Jose Rusca · Ing. Federico Stulich · Ing. Francisco De Grandis

Mapa de la unidad

La unidad conecta tres ideas: cómo viven las variables, cómo las funciones pueden tratarse como valores y cómo esas funciones se usan para procesar arreglos.

Tema	Idea central
1. Alcance	Dónde existe una variable: función o bloque.
2. Funciones anónimas	Funciones sin nombre que pueden guardarse, pasarse o devolverse.
3. Arrow functions	Forma compacta de escribir funciones, con diferencias importantes.
4. Callbacks	Funciones que otra función recibe como parámetro y luego invoca.
5. Closures	Una función interna conserva acceso al ámbito de la función exterior.
6. Spread	Expande los elementos de un iterable donde se esperan elementos o argumentos.
7. Arreglos	Creación y métodos para recorrer, transformar, buscar, unir y modificar arrays.

Conexión importante Los callbacks suelen escribirse como funciones anónimas o arrow functions, y varios métodos de arreglos reciben justamente ese tipo de funciones.

1. Variables y alcance

El alcance (scope) indica desde qué parte del código una variable puede ser utilizada.

REGLA DEL MATERIAL

Declaración	Alcance	Consecuencia práctica
var	A nivel de función	Si se declara dentro de un if, sigue siendo visible dentro del resto de esa función.
let	A nivel de bloque	Solo existe dentro del bloque { ... } donde fue declarada.
const	A nivel de bloque	En este material, se destaca el mismo alcance de bloque que let.

EJEMPLO: VAR

```
function ejemploVar() {
  var var1 = 10;
  if (var1 === 10) {
    var var2 = 40;
  }
  console.log(var2); // accesible dentro de la función
}
```

EJEMPLO: LET

```
function ejemploLet() {
  let let1 = 10;
  if (let1 === 10) {
    let let2 = 20;
  }
  console.log(let2); // error: let2 quedó dentro del bloque if
}
```

Para recordar var mira el límite de la función; let y const miran el límite de las llaves del bloque.

2. Funciones anónimas

Son funciones sin nombre propio. El material las presenta como útiles cuando queremos pasar una función como parámetro o devolver una función desde otra función.

```
var respuesta = function (nombre) {
  console.log("Hola " + nombre + "! Soy una funcion anonima");
};

respuesta("Juan");
```

Idea clave La función no tiene un nombre después de function. Sin embargo, puede guardarse en una variable (respuesta) y llamarse mediante esa variable.

3. Funciones flecha (arrow functions)

Son una forma compacta de escribir funciones. El material remarca tres diferencias frente a las funciones clásicas: no crean su propio contexto this, no se usan como constructor y pueden tener retorno implícito.

PATRONES DE SINTAXIS

```
() => { /* instrucciones */ }

(a, b, c) => {
  let res = a + b + c;
  return res;           // retorno explícito
}

(a, b, c) => a + b + c; // retorno implícito

a => a + 10;             // un parámetro: paréntesis opcionales
```

Llaves vs. sin llaves Con un bloque { ... }, el ejemplo usa return explícito. Sin llaves, la expresión se devuelve de forma implícita.

4. Callbacks

Un callback es una función que se pasa como parámetro a otra función. La función receptora puede invocarla cuando la necesita.

EJEMPLO DIRECTO

```
setTimeout(function () {
  console.log("Saludo en 1 segundo.");
}, 1000);
```

Aquí, la función anónima es el callback que `setTimeout` recibe como argumento.

MISMA FUNCIÓN RECEPTORA, DISTINTO COMPORTAMIENTO

```
let presentacionFormal = (nombre, apellido) =>
  "En este acto, presentamos a " + nombre + " " + apellido;

let presentacionInformal = (nombre, apellido) =>
  "Bienvenido " + nombre + " " + apellido;

function presentarPersona(nombre, apellido, funcionPresentacion) {
  console.log(funcionPresentacion(nombre, apellido));
}

presentarPersona("Juan", "Garcia", presentacionFormal);
presentarPersona("Juan", "Garcia", presentacionInformal);
```

Lectura conceptual `presentarPersona` no decide cómo presentar: recibe esa decisión en `funcionPresentacion`. Eso es lo que hace flexible al callback.

5. Clausuras (closures)

Una clausura es una función que conserva referencias al estado que la rodeaba. Por eso, una función interior puede seguir accediendo al ámbito de una función exterior.

EJEMPLO BÁSICO

```
function creaFunc() {
  var nombre = "Programacion Objetos 2";

  function muestraNombre() {
    console.log(nombre);
  }

  return muestraNombre;
}

var miFunc = creaFunc();
miFunc();
```

`creaFunc` termina y devuelve `muestraNombre`. Cuando luego se ejecuta `miFunc()`, esa función todavía puede acceder a `nombre`, que pertenecía al ámbito exterior.

EJEMPLO DEL SUMADOR

```
function creaSumador(x) {
  return function (y) {
    return x + y;
  };
}

var suma5 = creaSumador(5);
var suma10 = creaSumador(10);

console.log(suma5(2)); // 7
console.log(suma10(2)); // 12
```

Qué queda “guardado” suma5 conserva la referencia al x que valía 5; suma10 conserva otra referencia al x que valía 10.

6. Sintaxis spread (...)

Spread expande un iterable -como un arreglo o una cadena- en un lugar donde se esperan varios argumentos o varios elementos.

PASAR ELEMENTOS COMO PARÁMETROS

```
function sum(x, y, z) {
  return x + y + z;
}

const numbers = [1, 2, 3];
console.log(sum(...numbers));
```

...numbers convierte conceptualmente [1, 2, 3] en tres argumentos separados para la llamada a sum.

CONSTRUIR UN ARREGLO CON ELEMENTOS DE OTROS

```
var valores = [1, 2, 3];
var numeros = [4, 5, 6];
var totales = [...valores, ...numeros];
```

Lectura rápida ... no es “el arreglo”: es la expansión de sus elementos en el lugar donde se usa.

7. Arreglos

El material muestra tres formas de crearlos y luego una colección de métodos. Muchos de esos métodos reciben una función para procesar cada elemento.

CREACIÓN

```
let frutas = ["Manzana", "Banana"];
let frutas = new Array("Manzana", "Banana");
let frutas = new Array(10); // arreglo de longitud 10 sin items cargados
```

Cómo estudiar los métodos En vez de memorizar 16 nombres aislados, conviene agruparlos por intención: recorrer/transformar, buscar, unir/convertir y modificar.

7.1 Recorrer, transformar y resumir

Método	Qué hace	Ejemplo mínimo
forEach	Ejecuta una función una vez por cada elemento.	array1.forEach(element => console.log(element));
map	Crea un nuevo array con el resultado de aplicar una función a cada elemento.	dobles = numeros.map(x => x * 2);
filter	Crea un nuevo array solo con los elementos que cumplen una condición.	result = words.filter(word => word.length > 6);

Método	Qué hace	Ejemplo mínimo
reduce	Aplica una función reductora a los elementos y devuelve un único valor.	<code>array1.reduce((acc, actual) => acc + actual);</code>

7.2 Buscar y comprobar

Método	Qué devuelve	Ejemplo mínimo
includes	Indica si el array contiene un elemento. Devuelve true o false.	<code>pets.includes("cat") // true</code>
find	Devuelve el primer valor que cumple una función de prueba.	<code>array1.find(x => x > 10) // 12</code>
findIndex	Devuelve el índice del primer elemento que cumple la prueba; si no hay, -1.	<code>array1.findIndex(x => x > 13) // 3</code>
indexOf	Devuelve el primer índice de un valor dado; si no está, -1.	<code>array.indexOf(7) // -1</code>

7.3 Unir, copiar y convertir

Método	Qué hace	Ejemplo mínimo
concat	Une dos o más arrays y devuelve uno nuevo; no cambia los existentes.	<code>array1.concat(array2)</code>
slice	Copia una parte del array desde inicio hasta fin (fin no incluido); no modifica el original.	<code>nombres.slice(1, 3)</code>
join	Une los elementos en una cadena usando un separador.	<code>elements.join("-")</code>

7.4 Métodos que cambian el array

Método	Efecto	Ejemplo mínimo
pop	Elimina y devuelve el último elemento; cambia la longitud.	<code>plants.pop()</code>
push	Agrega uno o más elementos al final y devuelve la nueva longitud.	<code>animals.push("cows")</code>
shift	Elimina y devuelve el primer elemento; modifica la longitud.	<code>peces.shift()</code>
splice	Elimina y/o agrega elementos en una posición.	<code>months.splice(1, 0, "Feb")</code>
sort	Ordena el array localmente y devuelve el array ordenado.	<code>valores.sort((a,b) => a - b)</code>

Ojo con sort() En el ejemplo, sort() sin comparador no da el orden numérico esperado; para números se usa (a, b) $\Rightarrow$ a - b.

Repaso express

Si podés explicar estas frases sin mirar, ya tenés la idea central de la unidad:

- **var:** alcance de función.
- **let / const:** alcance de bloque.
- **función anónima:** función sin nombre, útil para guardarla, pasarla o devolverla.
- **arrow function:** sintaxis compacta; sin this propio; puede tener retorno implícito.
- **callback:** función pasada como parámetro para que otra función la use.
- **closure:** función que conserva acceso al ámbito exterior donde fue creada.
- **spread (...):** expande elementos de un iterable como argumentos o elementos.
- **forEach:** recorre.
- **map:** transforma y crea otro array.
- **filter:** selecciona y crea otro array.
- **reduce:** reduce a un único valor.
- **find / findIndex:** encuentra valor / índice según una condición.
- **includes / indexOf:** comprueba presencia / busca índice de un valor.
- **concat / slice:** unen o copian sin modificar el original, según el material.
- **pop / push / shift / splice / sort:** modifican el array original.
- **join:** convierte los elementos en una cadena.

Mini flujo mental para callbacks + arrays

```
const numeros = [1, 5, 10, 15];

const dobles = numeros.map(x => x * 2);
// map recibe como callback: x => x * 2
// y construye un nuevo array con cada resultado
```

Idea integradora Variables definen el estado; funciones pueden viajar como valores; callbacks llevan comportamiento; closures conservan contexto; y los métodos de arrays usan todo esto para procesar colecciones.

Fin del resumen

Siguiente tema indicado por el material: definición de clases y objetos en JavaScript.